Server-side Development with Node.js

Table of Contents

Sessions

SESSION 1

INTRODUCTION TO SERVER-SIDE DEVELOPMENT

Learning Objectives

In this session, students will learn to:

- Outline the client-server architecture
- Explain the significance of server-side programming
- Explain a Web server and its architecture
- Describe Node.js, its features, and its installation process

Today, Web applications have become an integral part of everyone's lives. Web applications are used to perform tasks such as shopping for various items, buying groceries, maintaining health data, booking travel tickets and hotels, and banking. These Web applications work on the client-server architecture model. In this architecture, data processing happens on the server, so the load on the users' device, such as a computer or mobile device, is minimal.

The client-server architecture provides security features, such as login authentication, to prevent unauthorized access to users' data. It also provides personalization features, which personalize the data displayed based on the user preferences. This feature enhances the user experience with the Web application. For the Web application to work, server-side programming is employed to create programs that process the user requests and respond to user actions.

This session will provide an overview of the client-server architecture and the importance of server-side programming in the smooth execution of Web applications. It will also introduce Node.js, which is a server-side programming environment that is widely used to develop Web applications. Finally, it will explain the steps to install Node.js.

1.1 What is Client-Server Architecture?

Consider that a user goes to an ATM to withdraw some cash. The user inserts the card into the ATM machine, which, in turn, reads the card, and prompts the user to enter the PIN. When the user enters a valid PIN, the user can perform various tasks such as viewing the balance amount in the account or withdrawing an amount. When the user selects the cash withdrawal option, the user is prompted to enter the amount required. If that amount is more than the amount available in the account, the ATM machine will display an error. If the amount is less than the amount available in the account, the required cash will be dispensed. Multiple users use the same ATM machine to access accounts in various banks.

In this case, the user interacts with the ATM machine. Behind the scenes, there are a number of processes happening. When the user enters the card, the ATM machine reads the card and sends the card details to the bank's server. The server, in turn, checks if the card is valid. If the card is valid, the server sends a signal to the ATM machine, which then prompts the user for the PIN. After the user enters the PIN, the PIN is sent to the server for validation. Only if the server sends a signal stating that PIN is valid, the ATM machine will display various options which user can use to interact with the bank account. When the user wants to withdraw cash and enters amount to withdraw, that amount is sent to the server to check if there is enough balance in the account. Based on the response that the server sends, the ATM machine dispenses the amount or gives an error.

Here, the ATM machine and the banks' server together form the client-server architecture.

Figure 1.1 depicts the client-server architecture.

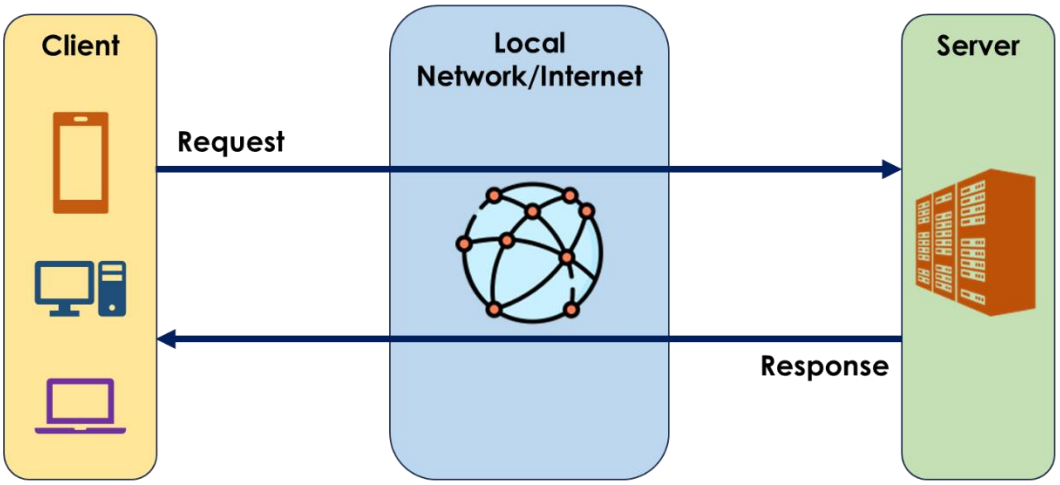


Figure 1.1: Client-Server Architecture

As shown in Figure 1.1 the main components of client-server architecture are as follows:

Client	Server	Network
Known as workstations or frontend. Used by users to provide instructions for performing various tasks. Connects with the server to deliver the user instructions and pass on the responses from the server to the user. Is governed by policies defined by the server. Can be desktop, laptop, tab, or mobile.	Known as backend. Is a centralized repository of files, programs, databases, and network policies. Has large storage space and memory to cater to requests coming in from several clients. Can be a file server, mail server, database server, Web server, or domain controller.	Is the medium through which clients and servers connect. Is made up of various networking devices that facilitate the communication between clients and server. Includes networking devices such as hubs, repeaters, bridges, and routers.

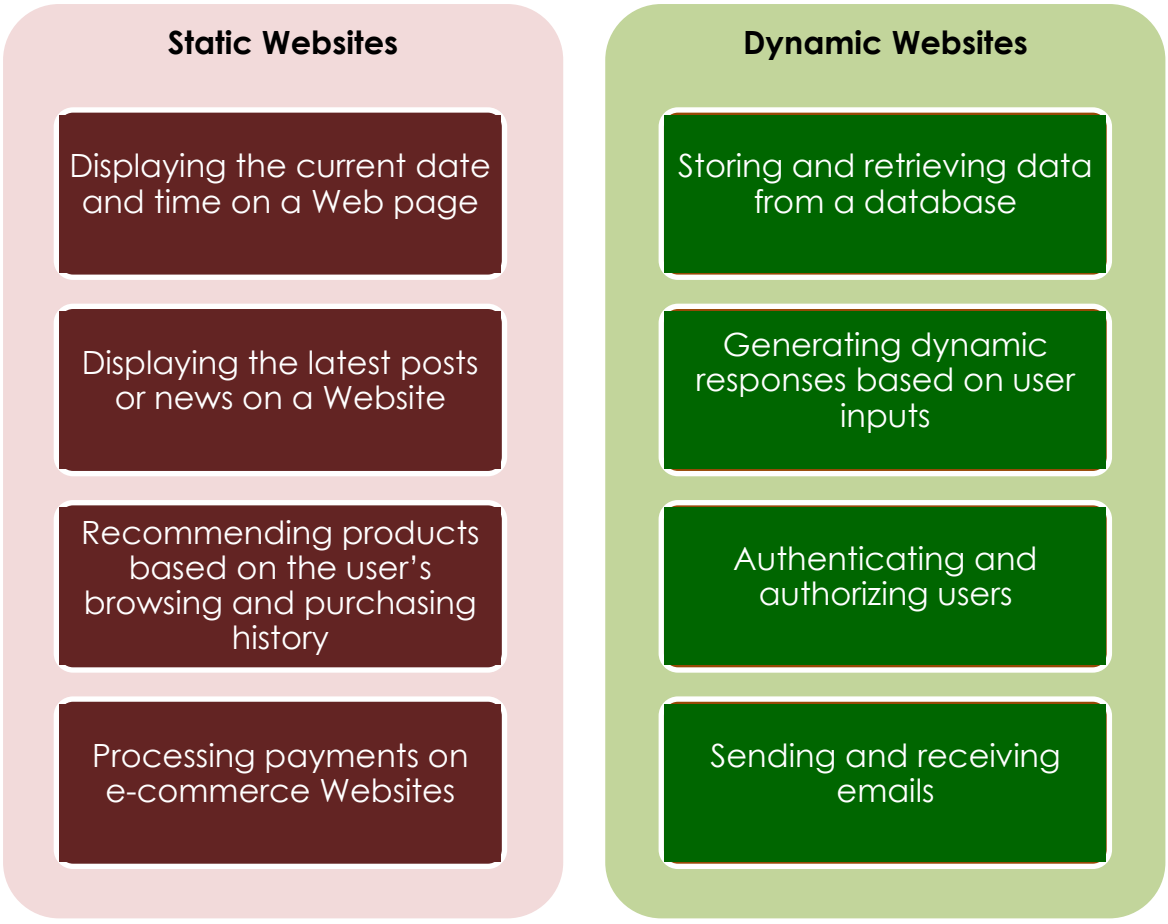
1.2 Server-side Programming

As discussed already, the server performs tasks that it receives as instructions from users through the client. To perform these tasks, various programs run on the server.

Developing these programs that run on the server is termed **server-side programming**.

To understand this better, consider the example of the ATM. There are programs that run on server for authenticating users' card and PIN, checking balance in the account, and updating the balance in the account after the cash withdrawal. This is also referred to as back-end processing. The user is not required to know what is happening on the server. Users are only required to provide instructions through the client.

Server-side programming is used to add dynamic features to static Websites or to create dynamic Web pages. Examples of server-side programming uses are as follows:



Server-side Technologies/Programming Languages

Various programming languages and technologies can be used to create server-side programs.

Some of these are as follows:

Python	It is a simple programming language that can be used to create Web applications. It is also extensively used in the field of data science and machine learning.
PHP	It is a scripting language widely used in the development of Web applications. It provides a large library of functions and classes that help make the applications fast and efficient.
Java	It is a programming language that can be used to create reliable and scalable Web applications. It is widely used to develop enterprise applications.
Node.js	It is a JavaScript runtime environment that facilitates developers to use JavaScript for server-side programming. It is lightweight and scalable. It is often used for building real-time Web applications.
Ruby	It is a dynamic programming language that is expressive and readable. It is popularly used for the development of Web applications.
C#	It is a general-purpose programming language that is used to develop Windows-based applications and Web applications.

Advantages and Disadvantages of Server-side Programming

Table 1.1 describes the advantages and disadvantages of server-side programming.

Advantages		Disadvantages	
Enhanced Security	Server-side programs are run on the server and never sent to the client. Therefore, the code cannot be intercepted or exploited by an attacker. Even if users	**Complexity**	Server-side programs must be able to handle a variety of tasks, such as database interaction, user authentication,

Advantages		Disadvantages	
	try to view the code, they can only view the client-side code and not the server-side code. This feature enhances the security of the Website or Web application.		and error handling. This requirement can make the program development complex.
High Scalability	Server-side programs can be scaled up to handle the increasing number of users and requests. This feature makes the server-side programs ideal for enterprise applications or Websites that experience heavy user traffic.	**Performance Overhead**	The server is required to process the data before sending the response to the client. This activity can add some performance overhead to a Website or Web application.
High Performance	Server-side programs run on the server. As a result, not all the code is downloaded to the client for execution. In addition, complex activities are completed in less time and steps. This feature improves the performance of the Website or Web application.	**Cost**	Applications that use server-side programs may require high-end and expensive servers to handle the data processing activities. This requirement can increase the cost of implementation of the Website or the Web application.
Adaptive Flexibility	Server-side programs can be used to implement a wide range of applications that provide several features such as generating dynamic	**Vulnerability**	Server-side programs can be vulnerable to security attacks, such as SQL injection or Cross-Site

Advantages		Disadvantages	
	content, authenticating users, and interacting with databases. In addition, server-side programs are not browser-dependent, so program developers do not have to worry about the browser versions.		Scripting (XSS). However, these vulnerabilities can be handled by applying best practices and using secure coding techniques.

Table 1.1: Advantages and Disadvantages of Server-side Programming

1.3 Introduction to Web Server

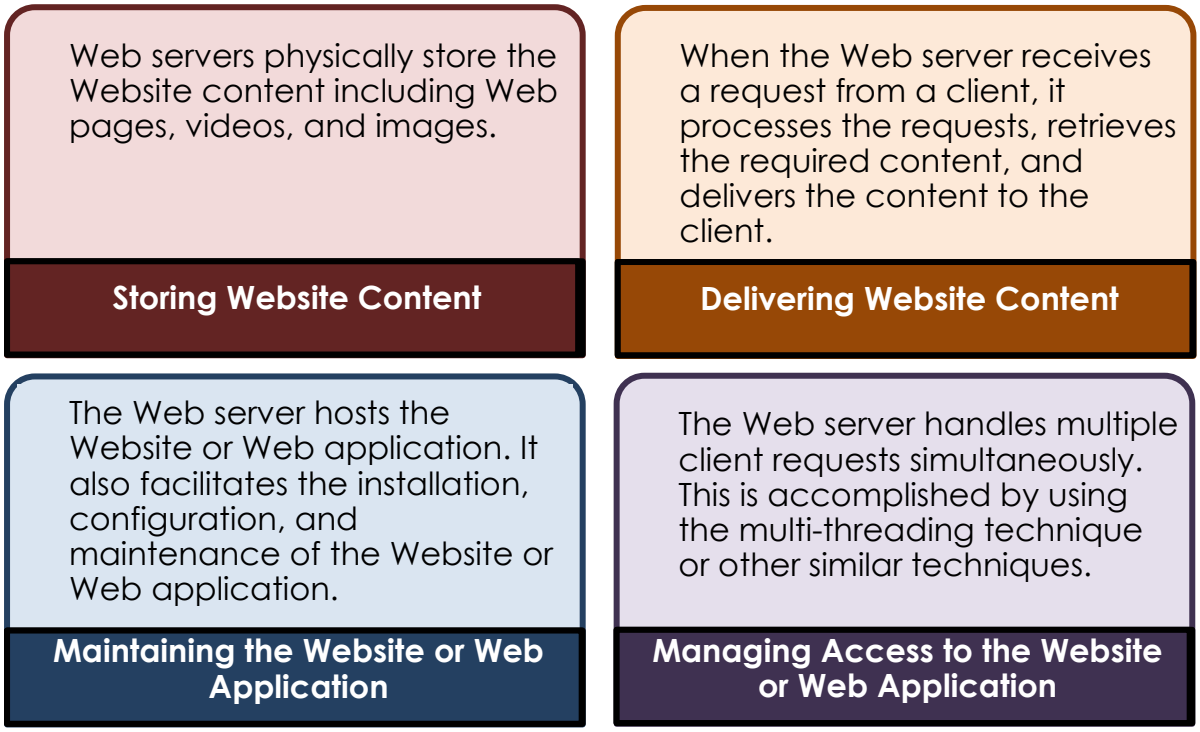
Servers can be of different types such as mail servers, DNS servers, and Web servers. Each server has a specific purpose.

A Web server is one that has the content required for a Website and has the logic for processing the data received from the client. A Web server stores files related to Websites. When a request for a Web page arrives through a browser, the Web server responds by returning the requested Web page to the Web browser.

A Web server has both hardware and software components to serve its purpose. The hardware connects the Web server to the network or Internet and facilitates the exchange of data with various clients.

The software component is responsible for processing the client requests and responding to them with the required information or data. The clients connect with a Web server using Hypertext Transfer Protocol (HTTP). This protocol is the guideline for the client for sending the requests and for the Web server for responding to the requests.

Functions that a Web server performs include:



1.3.1 Web Server Architecture

Web servers receive single or multiple requests from the clients simultaneously. The aim of the Web server is to handle these requests and respond to them with less wait time. Web server architecture is of two types based on the way the requests are handled:

- Concurrent approach
- Single-process-event-driven approach

Concurrent Approach

In the concurrent approach type of architecture, the Web server creates a thread for each request it receives. This facilitates simultaneous response to multiple requests. For example, consider that 10 friends plan to play online games and access the same gaming Website simultaneously. In this approach, the Web server that hosts the game will create ten separate threads, each thread serving one of the players. This ensures quick, real-time responses to the actions taken by the players, thereby enhancing player experience.

However, this approach requires large investments in hardware, such as CPU and memory, to handle multiple processing threads. The concurrent approach is best suited for Websites that experience heavy user traffic, such as social media, large e-commerce Websites, and gaming Websites.

Single-Process-Event-Driven Approach

In this single-process-event-driven approach, the Web servers use a single thread to handle all requests. The requests are handled in a non-blocking manner. For example, consider that there are ten users who are accessing a shopping Website. The users perform tasks such as browsing the catalog, viewing product details, and making purchases. In this approach, a single thread is used to handle user requests one-by-one as they come in, ensuring that one request processing does not block processes of other requests. The single-process-event-driven approach is best suited for Websites that experience low user traffic, such as small e-commerce Websites and Websites that provide information about companies.

1.3.2 Working of Web Servers

Figure 1.2 shows the working of Web servers.

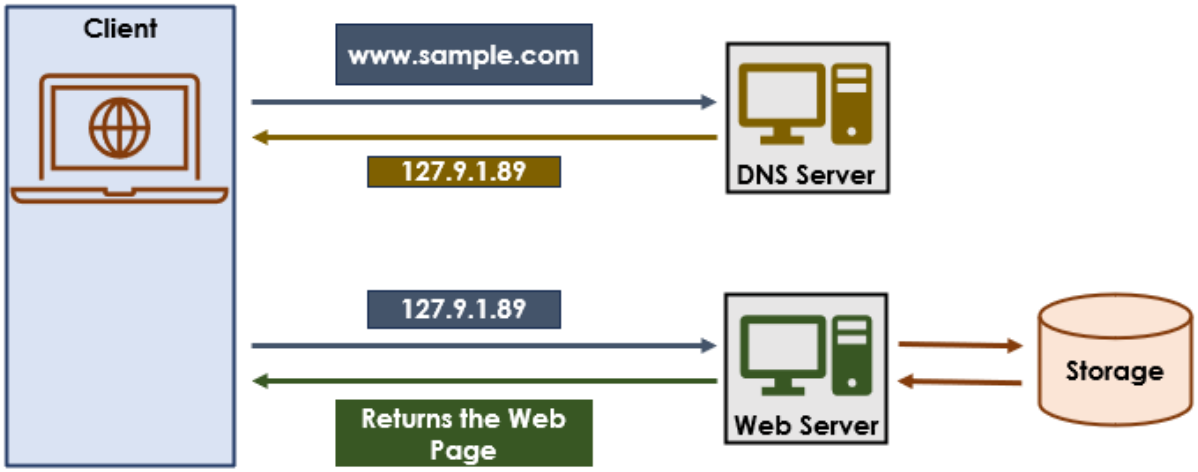


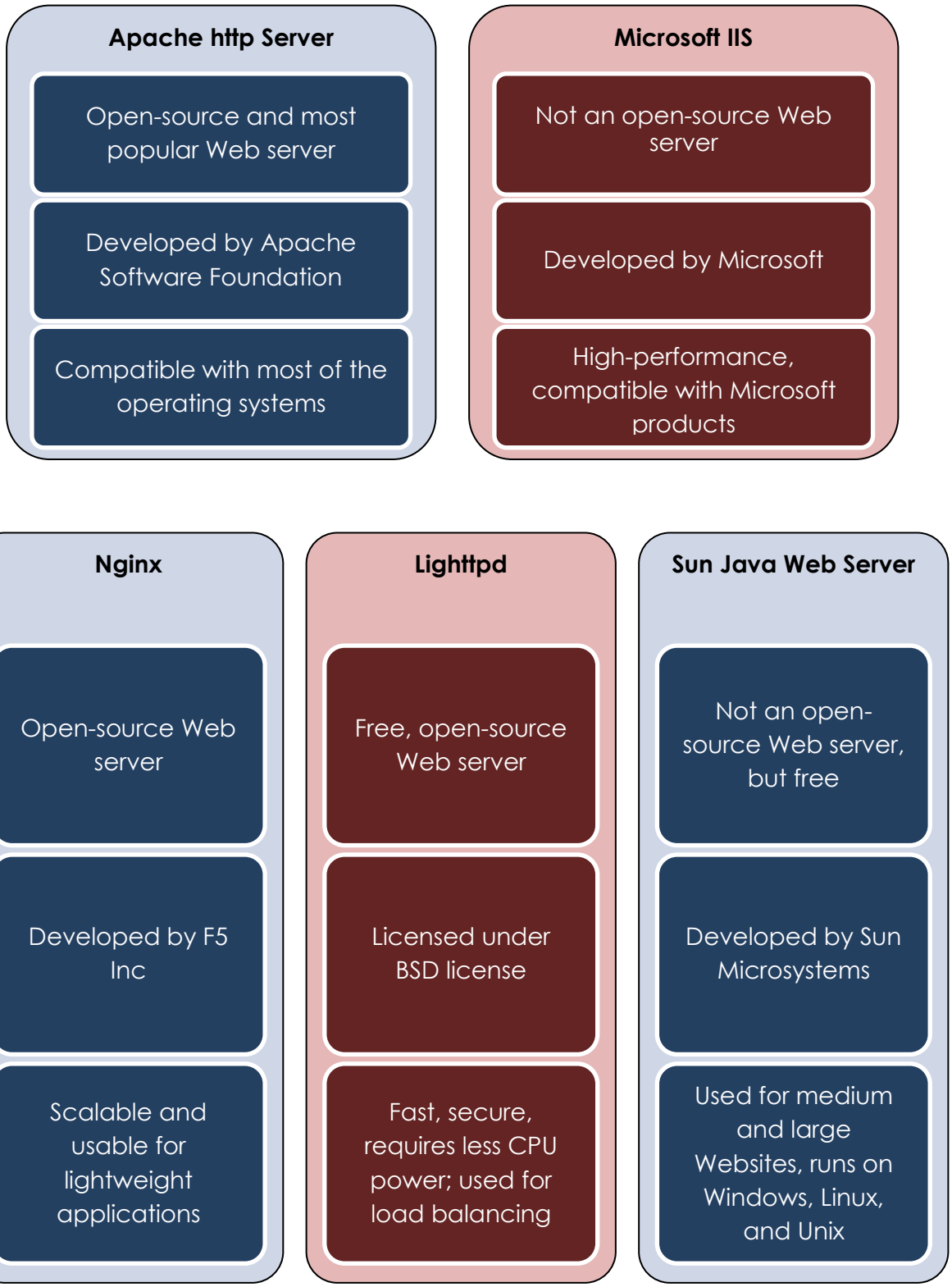
Figure 1.2: Working of a Web Server

As shown in Figure 1.2, a user enters a Universal Resource Locator (URL), for example, www.sample.com, in the Web browser on the client. The client sends this URL over the network to the DNS server. The DNS server checks for the URL in the database and returns the associated IP address to the client. The client then sends the IP address to the Web server. Finally, the Web server locates the file, image, or video in its storage and returns the requested content to the client. The client then displays the content received in the browser window.

1.3.3 Types of Web Servers

There are many types of Web servers. Each is designed for a specific purpose. Some are open-source, while some are not.

Some of the Web servers include:



1.4 Introduction to Node.js

Node.js is a runtime environment for developing server-side programs using JavaScript. It is used to develop Web applications. It is open-source and is

compatible across numerous platforms, including Windows, Linux, Mac OS, and Unix. It is built on Google Chrome's JavaScript engine (V8 Engine). It efficiently executes JavaScript code using a Just-in-Time compiler instead of an interpreter.

1.4.1 Features of Node.js

Node.js has some distinct features that make it the most preferred runtime environment for developing and implementing Web applications. These features are as follows:

- **It is easier for developers to adapt**

Most of the Web application developers are well-versed in the use of JavaScript coding. Node.js is also written in JavaScript, making it easier for the developers to adapt to it. JavaScript is also used to develop the front end for Web applications. A combination of JavaScript and Node.js helps developers to create full-stack Web applications by using only JavaScript to develop the client-side user interface and server-side programs. In addition, Node.js is compatible with various operating systems, such as Windows, Mac OS, Linux, or mobile platforms. This feature makes Node.js a preferred platform for developing and implementing Web applications.

- **It facilitates enhanced user experience**

Node.js has a non-blocking way of executing the instructions it receives and returning the response. It uses a single thread in which all the requests are queued and processed one after another as they come in. This way of handling requests ensures that there is no wait time for processing any of the requests. This asynchronous way of processing data provides enhanced performance of the Website or Web application. The fast performance, in turn, enhances the user experience when using the Website or Web application.

- **It reduces the cost of investing in server hardware**

Node.js uses an event-based approach. In this approach, an event loop handles the operations asynchronously. The event loop listens for events and then, calls the associated callback function. An event could be any action, for example, the clicking of a button by a user is an event. The callback function associated with the event provides the responses to these events. For the callback function to be executed, fewer resources and less memory on the server side are required. This feature ensures a reduction in the cost of investment in server resources.

- **It is a scalable runtime environment**

As the number of users accessing the Website or Web application increases, the processing power of the server must also be increased to handle the users.

Increasing the processing power in terms of memory, CPU, and other resources can be expensive. With Node.js, this scalability is in-built because it uses the event-driven approach of handling user requests. Applications running on Node.js are scalable as the environment provides the elasticity and flexibility to adjust or customize according to the demanding situations.

- **It facilitates quick data streaming**

Sequential data handling of input and output is termed data streaming. Node.js reads or writes chunks of data, processes it and sends it back without holding it in memory. This feature reduces the overhead of buffering data and makes the streaming process faster.

1.4.2 Node.js Installation

To install Node.js, perform these steps:

1. Open a browser and navigate to the URL: https://nodejs.org/en/download

The Web page opens as shown in Figure 1.3.

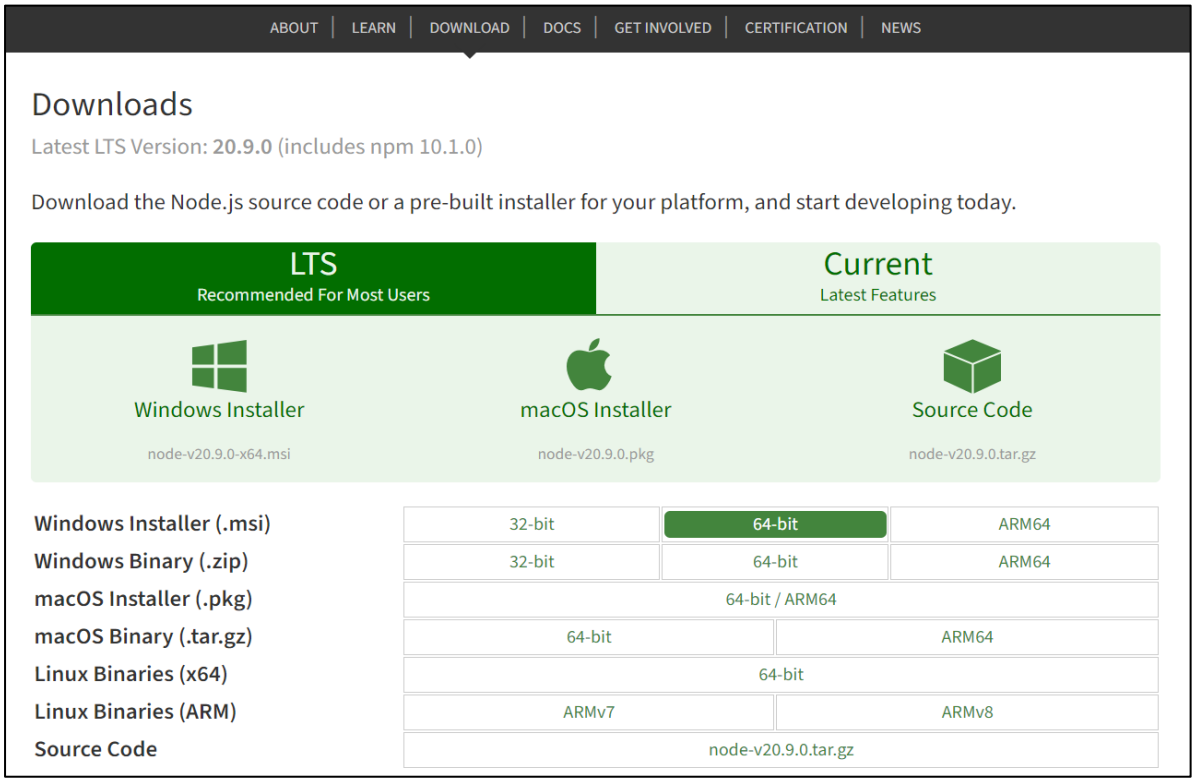


Figure 1.3: Downloading Node.js Installer

2. Based on the operating system installed on the computer, download the associated .msi file by clicking it.
3. On the computer, navigate to the folder where the file is downloaded.
4. To start the installation, double-click the .msi file.

The Node.js Setup wizard launches and the Welcome page is displayed, as shown in Figure 1.4.

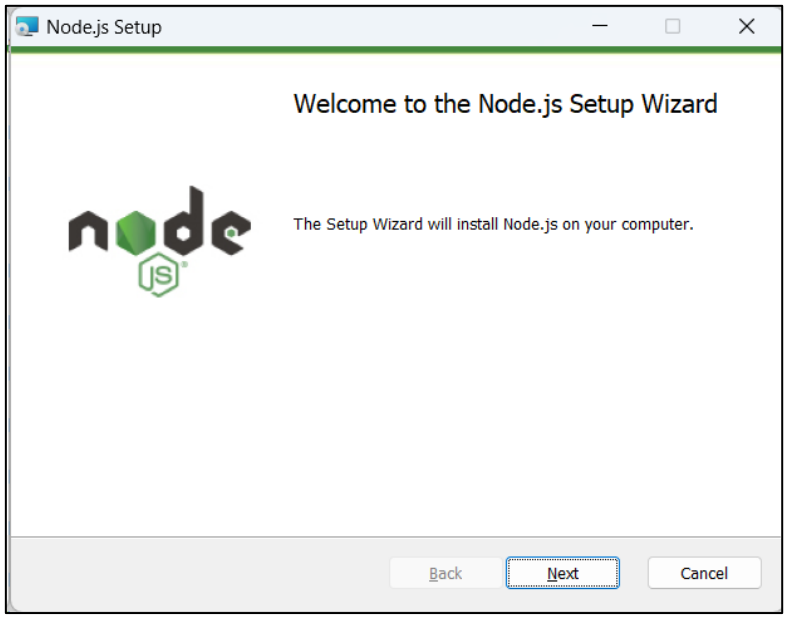


Figure 1.4: Node.js Setup Wizard – Welcome Page

5. On the Welcome page of the wizard, click **Next**.
   The End-User License Agreement page of the wizard opens.
6. On this page, select the **I accept the terms in the License Agreement** check box and click **Next**, as shown in Figure 1.5.

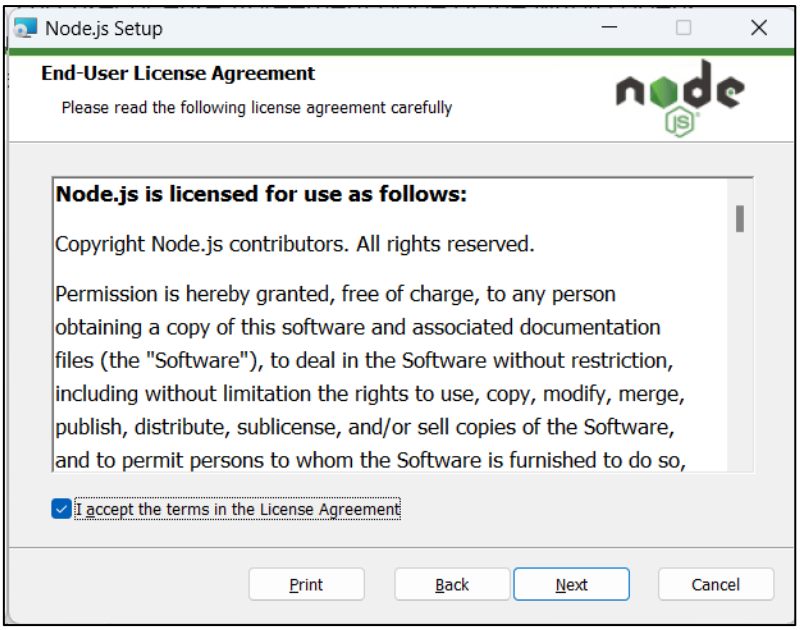


Figure 1.5: Accepting the End-User License Agreement

The Destination Folder page of the wizard opens, as shown in Figure 1.6.

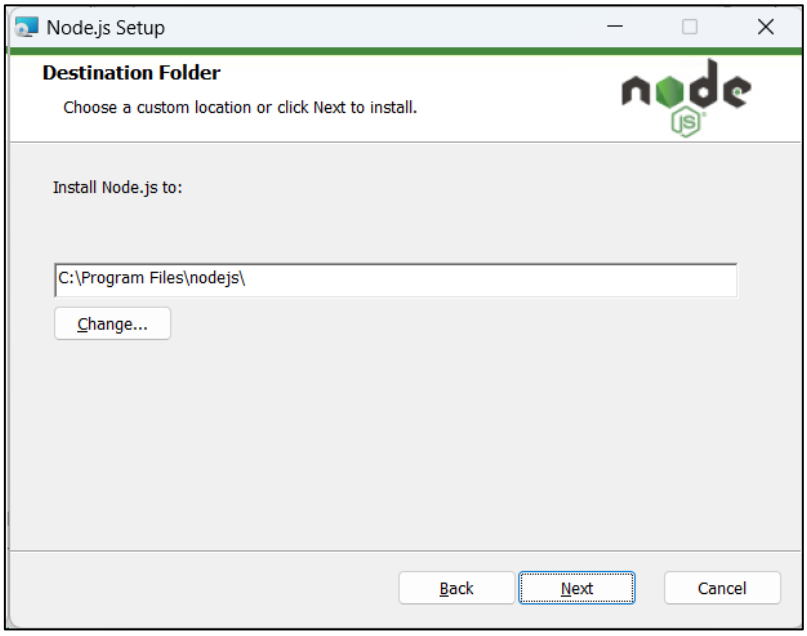


Figure 1.6: Selecting Destination Folder

7. To install Node.js on the default path, click **Next**.
   The Custom Setup page of the wizard opens, as shown in Figure 1.7.

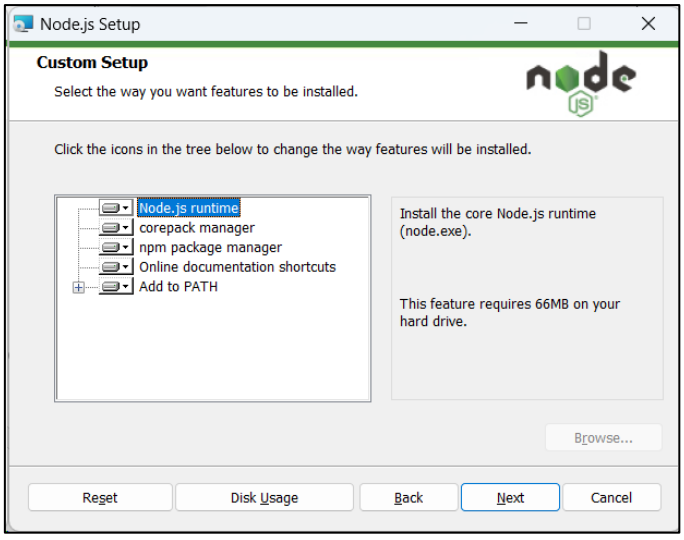


Figure 1.7: Selecting the Features to Install

8. To install all the features on the computer, click **Next**.
   The Tools for Native Modules page of the wizard opens, as shown in Figure 1.8.

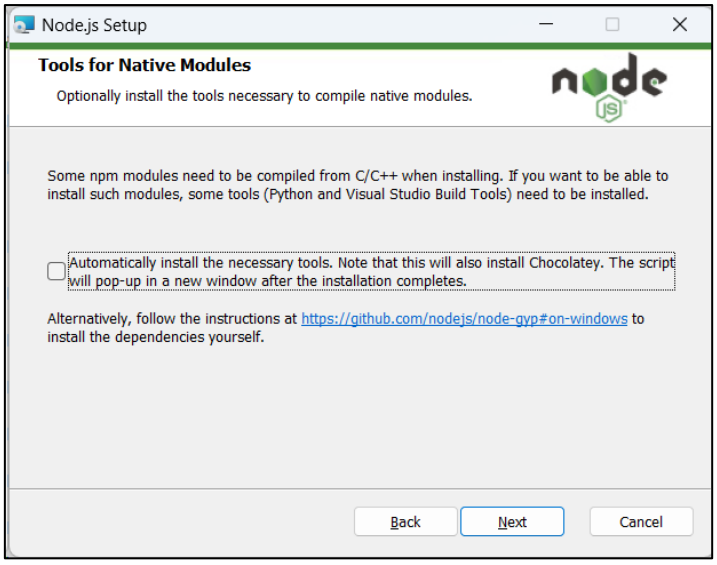


Figure 1.8: Installing Necessary Tools

9. To continue without installing the additional tools, click **Next**.
   The Ready to Install Node.js page of the wizard opens, as shown in Figure 1.9.

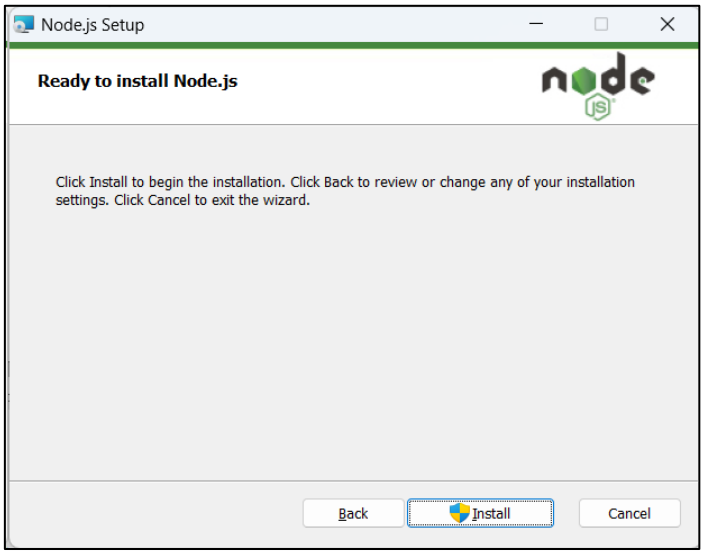


Figure 1.9: Installing Node.js

10. To install Node.js, click **Install**.
    Node.js is installed on the computer. After the installation is complete, the Completed Node.js Wizard page opens, as shown in Figure 1.10.

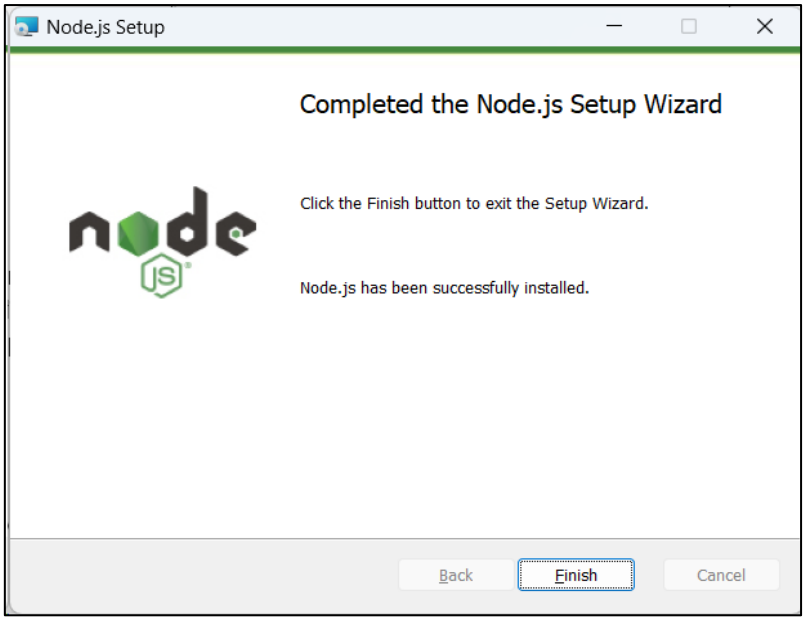


Figure 1.10: Completing the Installation

11. To close the wizard, click **Finish**.
12. To verify that Node.js has been installed, open a Command Prompt window.

13. At the command prompt, type the command as:

```
node -v
```

The version of Node.js that has been installed will be displayed as shown in Figure 1.11.

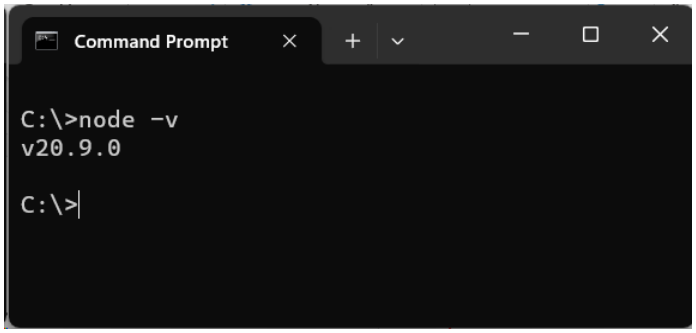


Figure 1.11: Checking Version of Node.js

To create scripts/programs for Node.js, one can use any text editor or Integrated Development Environment (IDE). Visual Studio (VS) Code is recommended for code development as it is free and also offers support to run and debug Node.js applications.

Developers can download and install VS Code using this link: https://code.visualstudio.com/

Set the **Path** environment variable using Control Panel to include the folder for Node.js as shown in Figure 1.12. The Path variable requires to be set only once.

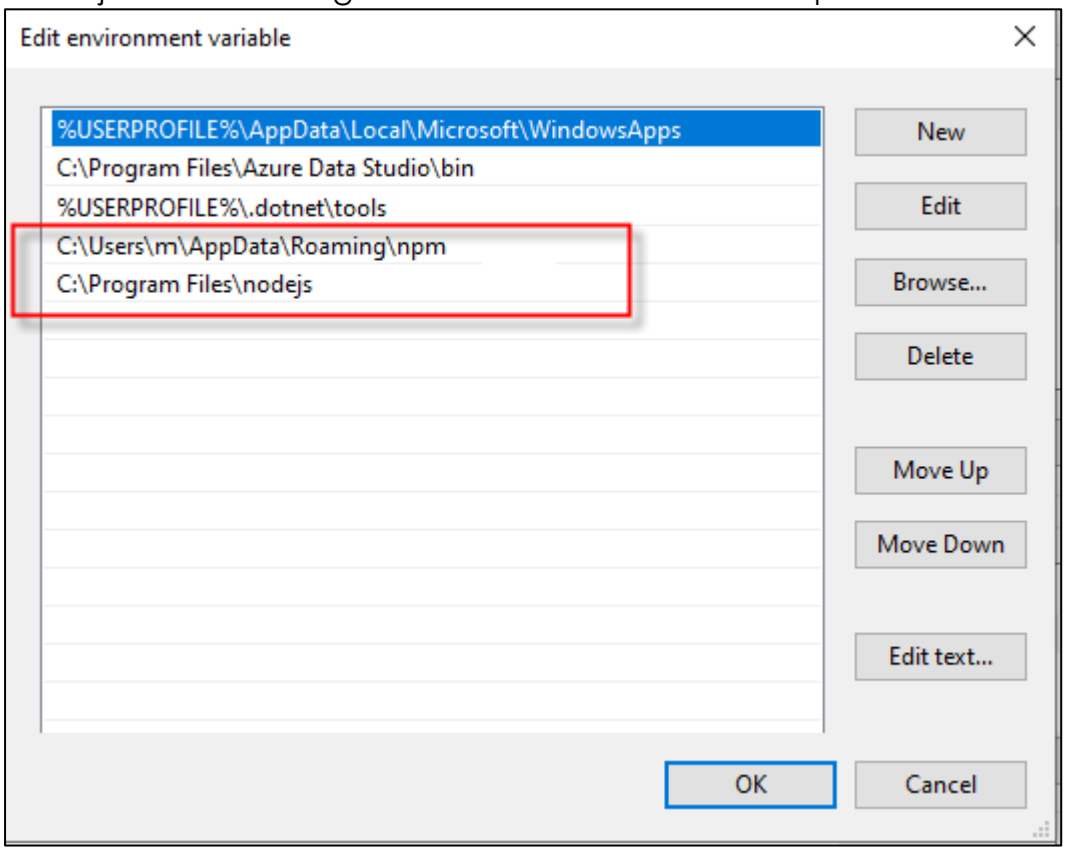


Figure 1.12: Setting Path for Node.js

Now, developers can launch VS Code and use it to write, run, and debug Node.js applications.

1.5 Summary

- Client-server architecture allows the server to serve requests from multiple clients.
- Client, server, and network are the components of the client-server architecture.
- The DNS server and Web server are the two servers that help in fetching the Web page when a request is made.
- Server-side scripting allows data processing to be done on the server side, thereby keeping the processing part hidden from the client.
- Java, PHP, Python, Node.js, Ruby, and C# are some of the server-side scripting tools.
- Node.js is a single-threaded, runtime environment for running JavaScript.
- Ease of working with JavaScript, scalability, and fast streaming of data makes Node.js a preferred runtime environment for Web applications.
- Visual Studio Code can be used as an IDE for Node.js applications.

Test Your Knowledge

1. A public library catalog system is managed using client-server architecture. Visitors search for books using a computer terminal. Results are displayed on the screen. In this library scenario, which function does the server perform?

    a) It allows visitors to search for books
    b) It displays book information on the screen
    c) It stores and manages the catalog data
    d) It serves as a client for the visitors

2. In Web development what is the difference between server-side and client-side programming?

    a) Code execution on the client's side is handled using server-side programming
    b) Process requests on the Web server are handled using server-side programming
    c) Server databases are managed using client-side programming
    d) Client-side programming controls the server's response to requests

3. A Web server architecture that can efficiently handle thousands of simultaneous connections is required for an online gaming platform. Which Web server architecture approach will be used for this situation?

    a) Blocking Approach
    b) Concurrent Approach
    c) Single-Process-Event-Driven Approach
    d) Non-blocking Approach

4. In concurrent approach how task is assigned to each thread?

    a) Each task is assigned to one thread
    b) Multiple tasks are assigned to a single thread
    c) Tasks are assigned to the threads based on the availability
    d) Based on the server configuration tasks are decided

5. What are the important features of Node.js?

    a) Asynchronous Nature
    b) Sharding
    c) Minimum Buffering
    d) Replication of Data

Answers to Test Your Knowledge

1	c
2	b
3	c
4	a
5	a, c

Try It Yourself

1. Install Node.js version 20.7.0 on a Windows system.

2. Verify the installation of Node.js on the Windows system.

3. Prepare a presentation with at least five points recommending why Node.js is good for server-side programming.